

AGENTTX: Causal Effect Transactions for Long-Running Coding Agents

Anonymous submission

Abstract

Coding agents execute long sequences of opaque tools that read, create, delete, and rewrite shared files. Existing choices leave an undesirable gap: direct execution exposes the host to every intermediate mistake, while per-call sandboxes and whole-session checkpoints either break trajectory state or discard useful work after a failure. We present AGENTTX, a runtime that treats an agent trajectory as a causal effect transaction. At tool boundaries, AGENTTX captures read, negative-lookup, write, and delete effects, constructs a cross-step dependency DAG, and keeps mutations in a shared unprivileged semisolate. A failed producer can be rolled back together with only its causal descendants; independent later work remains available. A commit frontier, policy engine, content-addressed snapshots, and a write-ahead log make selected effects durable without exposing unapproved state. Our Linux prototype is built on `try` and supports arbitrary subprocesses. Across 144 controlled recovery runs, causal rollback retained all independent work and removed all invalid descendants; temporal rollback retained 41% at 64 calls, while a dependency-capture ablation removed only 4% of invalid work. In a controlled real-agent replay study, causal retention avoided 1,336–2,891 tokens at the largest input. Persistent workers reduce the full 64-call path to 148 ms per step, 43% below naive per-call isolation, although it remains about $3\times$ slower than unsafe bare execution.

1 Introduction

Coding agents increasingly act as long-running systems programs. A single task may inspect a repository, edit several files, invoke compilers and tests, remove artifacts, revise a failed approach, and finally publish a subset of the work. The operating system sees a sequence of ordinary processes, but the user sees one adaptive trajectory whose intermediate effects should remain provisional. Today, every tool often writes directly to the host. A wrong cleanup command, generated configuration, or faulty refactor therefore becomes visible before the agent or user has approved it.

Sandboxing one command at a time prevents immediate pollution, but does not provide a transaction for the trajectory. Later tools must observe earlier speculative files, and repeating namespace and overlay setup on every call is expensive. A session sandbox preserves state, yet its native recovery unit is the entire session. Temporal checkpoints improve that unit to a prefix, but a rollback still discards every later action, including actions unrelated to the failure. Branch abstractions similarly select a whole branch rather than a non-contiguous subset within one branch [1, 5, 7].

The key observation is that time is not causality. Suppose an agent writes a faulty formatter, later generates a report by reading it, and independently updates release notes between the two. Recovery should remove the formatter and report while retaining the release notes. Doing so requires the runtime to discover effects of opaque commands, identify dependencies despite filesystem aliases, and reconstruct a state that never existed at a single point in time.

We propose an *agent effect transaction* (AET), in which tool calls are the visible boundaries and filesystem effects are the transactional objects. AGENTTX captures successful reads and negative lookups from system calls, derives writes and deletes from an overlay, and records a causal effect DAG. All changes remain in a shared semisolate until an explicit commit. On failure, AGENTTX computes the transitive causal closure and restores only paths written by that closure. It fails closed if a retained effect overlaps a reconstructed path. A commit frontier independently advances approved effects while later steps remain speculative.

We implement AGENTTX as a roughly 3.5 K-line Python runtime on Linux. It builds on the unprivileged semisolate and effect-control primitives of `try` [2]; our contribution is the trajectory-level ledger, causal recovery, historical frontier materialization, policy enforcement, and durable commit recovery. A persistent worker amortizes namespace setup, while incremental content-addressed snapshots preserve pre-step states. A commit WAL restores the host and upper layer after an interrupted multi-path commit.

Our current evaluation provides three main findings. First,

full AGENTTX is substantially faster than naive per-call isolation on a 64-call coding workload, although full tracing still costs about $3 \times$ bare execution. Second, controlled real-overlay DAG experiments show that full causal recovery retains 100% of independent work and removes 100% of invalid descendants; either temporal recovery or missing dependency capture loses one side of this goal. Third, the retained work translates into measurable model cost: at 48 entries per document, AGENTTX avoids 1,336 replay tokens relative to an optimistic checkpoint and 2,891 relative to whole-branch abort.

This paper makes the following contributions:

- It defines the AET abstraction, including an effect DAG, a causal rollback set, and a commit frontier for long adaptive trajectories.
- It presents an unprivileged runtime that combines opaque-command effect capture with selective reconstruction, durable snapshots, policy, and a crash-recoverable commit path.
- It evaluates both runtime and semantic outcomes, including optimization history, dependency ablation, 144 controlled recovery runs, real-agent recovery, replay tokens, worker crashes, session reload, and concurrency.

2 Background and Problem

2.1 Agent Trajectories and Effects

We model an agent trajectory as an ordered sequence of tool calls $T = \langle s_0, s_1, \dots, s_n \rangle$. A tool call can be a structured API, a shell script, a compiler, or an arbitrary third-party program. The agent framework knows the boundary and requested arguments, but generally does not know the complete set of files read or changed by descendants of the process. The OS observes these effects but lacks the trajectory semantics needed to decide which state is approved or which later work depends on a mistake.

For each step, AGENTTX records effects E_i over workspace paths. Effects are successful reads (R), negative lookups (N), writes (W), and deletes (D). Negative lookup is important: if step i observes that a generated file does not exist and step j later creates it, j may invalidate the assumption made by i . Directory ancestry is also relevant because changing a parent can alter the meaning or reachability of a child.

2.2 Isolation and Recovery Granularity

Table 1 separates isolation from recovery semantics. Bare execution has neither. A per-call semisolate can discard one command, but it must explicitly carry successful state between calls and repeatedly pays setup cost. A session sandbox stages a whole trajectory but naturally commits or aborts the session. Temporal checkpoints retain a prefix. None of these selects a producer and its non-contiguous descendants while preserving independent later work.

Semisolates and `try` provide powerful language-agnostic capture and optional application of effects [2]. AGENTTX deliberately reuses this substrate. The missing abstraction is orchestration across a long trajectory: assigning step identities, recovering dependency edges, preserving historical states, selecting a non-contiguous rollback set, and durably advancing a commit frontier.

2.3 Goals and Scope

AGENTTX targets filesystem effects of trusted but fallible coding agents. It must keep the host clean before approval, preserve normal POSIX tools, recover independent work, survive runtime crashes, and require no privileged daemon. The current prototype does not mediate irreversible network or cloud effects, does not capture process memory, and is not a security boundary against an adversarial process. These boundaries distinguish recovery from confinement.

3 Observations and Motivation

3.1 Why Simple Isolation Is Insufficient

We first run the same deterministic 64-call coding workload under six modes. The workload includes repository exploration, a multi-file refactor, an injected failing CI path, an artifact derived from the faulty file, independent documentation and configuration changes, and a repair suffix. Bare execution is fastest at 49.7 ms per step but modifies the host immediately. Per-call `try` costs 260.7 ms per step. Full AGENTTX costs 148.5 ms per step and keeps the host clean, while the no-trace ablation costs 62.6 ms but cannot recover read-derived descendants correctly.

Figure 1 records the implementations before each hot-path optimization. Declared effects for trusted tools avoid unnecessary tracing; reusable command scripts, deferred blob collection, and direct script execution remove smaller repeated costs. A persistent worker reduces 393.6 to 151.5 ms per step by amortizing namespace and overlay setup. Incremental snapshot replay reduces the snapshot stage from 0.384 to 0.158 s, but we do not present that substage result as an equivalent endpoint speedup.

3.2 Why Temporal Recovery Wastes Work

Consider steps $s_0 : W(a)$, $s_1 : R(a), W(b)$, and $s_2 : W(c)$, followed by a failure attributed to s_0 . A whole-session abort loses a , b , and c . A temporal rollback to before s_0 has the same outcome. The desired state removes a and derived b but retains independent c . Longer trajectories interleave many such branches, so the useful work lost by a time boundary grows with the distance between failure and diagnosis.

This distinction affects more than files. Recreating valid artifacts requires the agent to reread context, regenerate content,

Mode	Speculative unit	Native recovery unit	Later independent work	Host before approval
Bare	none	none	survives by immediate mutation	polluted
Per-call semisolate	one call	one call	no continuous shared state	clean
Session semisolate	trajectory	whole session	discarded on abort	clean
Temporal checkpoint	trajectory prefix	suffix after checkpoint	discarded with suffix	clean
AGENTTX	trajectory effect DAG	failed causal closure	retained when non-overlapping	clean

Table 1: Isolation does not imply trajectory-level causal recovery. The checkpoint and whole-session rows describe recovery granularity; they are not claims about a particular external artifact.

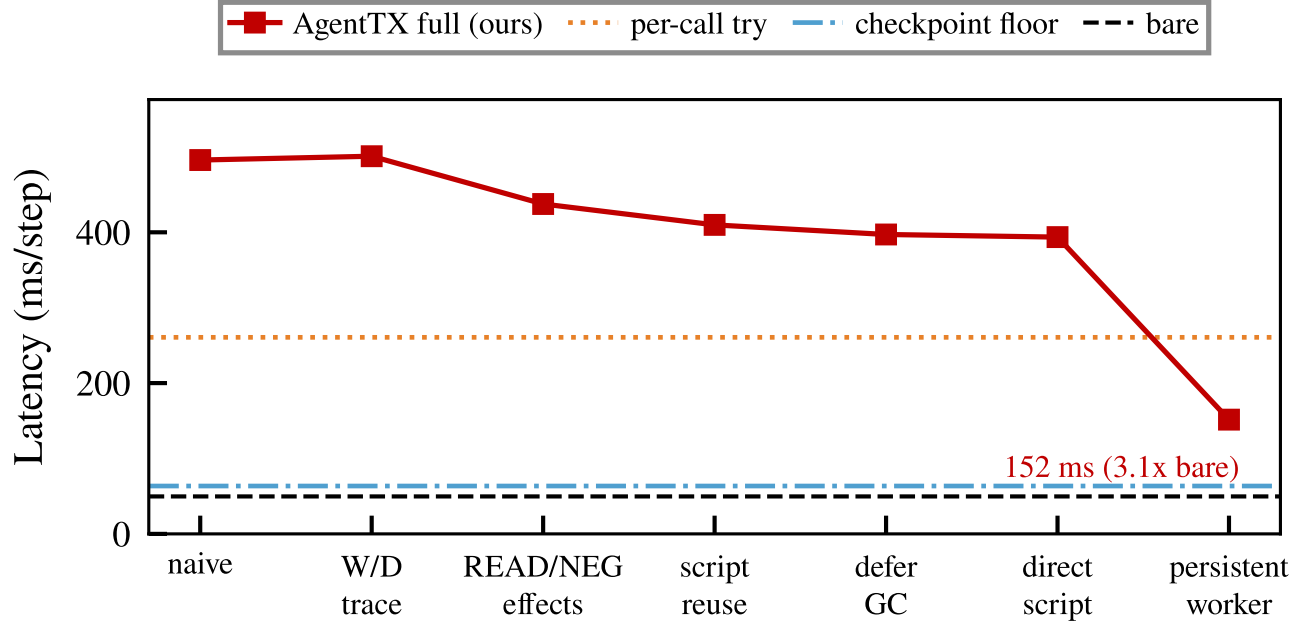


Figure 1: Optimization history on the long deterministic workload. The red curve records the full path after each preserved source iteration. A persistent `try` worker removes repeated namespace setup and is the largest endpoint improvement. Values are VM-local; the final point is still above unsafe bare execution.

invoke tools, and rerun checks. File retention is therefore a mechanism-level metric; avoided replay tokens are one direct user-visible consequence.

3.3 Three Systems Challenges

Dependency discovery. Opaque commands hide their producer-consumer relationships. Missing an edge keeps an invalid descendant; adding every temporal edge degenerates to suffix rollback.

Object identity. Paths are names, not stable objects. Directory hierarchy, symlinks, hard links, renames, and OverlayFS copy-up can make lexical comparison either miss or invent dependencies.

Selective reconstruction. The desired post-recovery state may never have existed at a point in time. The runtime must

undo a non-contiguous set, preserve later independent effects, handle repeated writes and deletions, and remain recoverable across a partial host commit.

These challenges motivate a trajectory abstraction in which dependency capture and state reconstruction are designed together.

4 Agent Effect Transactions

4.1 Overview

Figure 2 shows the lifecycle. A session owns a shared upper layer, a pre-step snapshot store, a ledger, and durable metadata. Each command starts from the current merged view. After execution, AGENTTX captures effects, adds one ledger node, persists the ledger, and leaves the upper layer speculative. The agent or user can request causal rollback or advance a commit frontier.

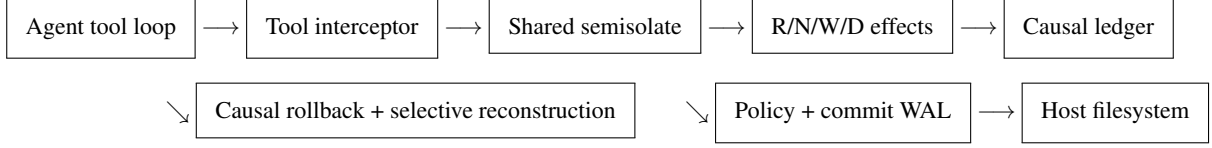


Figure 2: AGENTTX overview. Commands execute against one shared speculative view. The ledger determines causal recovery and the commit frontier; the host changes only after policy approval and WAL preparation.

4.2 Effect DAG

Each ledger node $s_i = (i, \text{tool}_i, E_i, P_i, \text{status}_i)$ contains effects and parent step identifiers. Define $\text{overlap}(p, q)$ when the normalized paths are equal or one is an ancestor of the other. For an active earlier step $j < i$, AGENTTX adds $j \in P_i$ when:

- s_j writes or deletes p , and s_i reads, negatively looks up, writes, or deletes an overlapping q ; or
- s_j negatively looks up p , and s_i later writes overlapping q .

The first rule captures read-after-write and write serialization. The second captures assumptions based on absence. Alias canonicalization may add another effect path before edge construction.

For a failed step f , causal rollback selects

$$C(f) = \{f\} \cup \{s_i \mid P_i \cap C(f) \neq \emptyset\}^*, \quad (1)$$

the transitive descendant closure among active, uncommitted steps. Temporal rollback remains available as the suffix $\{s_i \mid i \geq f\}$ for comparison and compatibility.

4.3 Selective Reconstruction

Let $W(C)$ be paths written or deleted by steps in $C(f)$. AGENTTX restores each path in $W(C)$ from the snapshot captured before the earliest selected step. Before reconstruction, it checks every effect of retained active steps. If a retained path overlaps $W(C)$, the runtime rejects the operation rather than guessing a merge. This fail-closed rule trades availability for a simple safety invariant:

After a successful causal rollback, no effect written by $C(f)$ is visible, and every non-overlapping retained effect remains visible in the speculative view.

The ledger marks selected nodes rolled back only after state reconstruction succeeds. Snapshots for invalidated steps can then be reclaimed.

4.4 Commit Frontier

The committed frontier F is the largest approved step identifier. Advancing to F' selects active W/D effects in $(F, F']$. Effects after F' remain in the upper layer. If later steps touch a selected path, AGENTTX uses the pre-later-step snapshot to materialize the historical version and then restores the later

speculative state. Policy is evaluated before any WAL or host materialization. Rolled-back holes do not block monotonic frontier progress.

5 Design and Implementation

5.1 Tool-Boundary Effect Capture

Writes and deletes are obtained by fingerprinting the unmounted upper layer before and after a command. Fingerprints include content and metadata, thereby capturing repeated `chmod`, `chown`, `touch`, `empty-directory`, `symlink`, and `whiteout` effects. Linux OverlayFS may encode deletions as character-device or `.wh.*` whiteouts; both forms are normalized to D effects.

Successful reads and `ENOENT/ENOTDIR` lookups are captured with `strace` [4]. The parser follows process creation, tracks each process working directory, resolves `*at` operations relative to file descriptors, and records both requested and returned paths when a symlink changes the pathname. Only workspace-local paths enter the ledger. Trusted structured tools can declare effects directly and bypass tracing. Unsupported or missing tracing fails closed unless the caller explicitly chooses the no-trace ablation.

5.2 Shared Semisolate and Persistent Worker

All calls in a session reuse one `try -N` upper layer, so a later command observes earlier speculative state. The optimized path starts a Python worker inside a persistent semisolate and exchanges length-prefixed requests and responses. A worker failure falls back to the original one-shot path for the current command; the next command starts a fresh worker. The launcher also synchronizes `$PWD` with the protected workspace because changing a subprocess `cwd` alone leaves a stale environment variable that upstream `try` may use as its chroot start directory.

5.3 Snapshots and Aliases

Before step i , the layer store records `before_i`. Regular files are stored as immutable content-addressed blobs and hard-linked into snapshots. After the first step, an incremental snapshot clones the prior tree and replays only paths changed by the previous call. Native whiteouts and mode-000 trees are

preserved. Causal rollback restores only selected logical paths; temporal rollback swaps the full upper tree to one pre-step image.

Path hierarchy is handled by the overlap predicate. For symlinks, AGENTTX resolves lexical ancestors in the merged host/upper view and records both alias and canonical effects. Hard links expose a substrate boundary: on the tested Linux 5.15 OverlayFS path, writing one lower hard-link name copy-ups and splits that inode, so a sibling alias still reads old content. Ledger-only canonicalization cannot repair semantics already lost below interception.

5.4 Durable Commit and Policy

Commit uses anchored include filters to materialize only selected paths. Directory and file metadata are restored after materialization. Before touching the host, a WAL saves the selected host paths, the upper layer, the target frontier, and the prior ledger. WAL phases distinguish prepared, applying, materialized, and committed states. On session load, AGENTTX either restores the prior host/upper state or finalizes a durably persisted frontier. Session metadata itself is written by same-directory atomic replace followed by file and directory fsync.

A shared allow/deny policy is enforced by the runtime, CLI, coding harness, and LLM agent. Denied paths fail before WAL preparation. Policy rules persist in session metadata, preventing a reload from bypassing the original approval boundary. The WAL makes recovery deterministic after a crash, but does not make a multi-path host update invisible to an external concurrent observer.

6 Evaluation

Our evaluation asks:

- What overhead does trajectory isolation and dependency capture add?
- Does causal rollback remove invalid work while retaining independent work, and are both the dependency graph and policy necessary?
- Can a live LLM use the recovery control plane, and does retained work reduce replay tokens?
- Does the optimized runtime survive worker failure, reload, long sessions, and concurrent agents?

6.1 Setup and Methodology

Experiments run on the project VM with Linux 5.15, the upstream `try` semisolate, OverlayFS, `strace`, and Python 3.11 for the evaluation environment. Deterministic workloads report repeats, p50, and p95 where available. Real-agent experiments use `deepseek-chat`. All correctness experiments inspect both the speculative merged view and the host before and after commit.

Mode	ms/step	Host clean
Bare	49.7	no
Per-call <code>try</code>	260.7	yes
Shared <code>try</code>	254.0	yes
Shared checkpoint	63.5	yes
AGENTTX no-trace	62.6	yes
AGENTTX full	148.5	yes

Table 2: Current 64-call deterministic workload. Lower latency alone does not imply equivalent recovery semantics.

Runnable baselines include bare, per-call `try`, session/shared `try`, shared checkpoint components, bubblewrap where applicable, full AGENTTX, and a no-read-tracing ablation. Temporal checkpoint and whole-branch/session abort in the retention and token studies are recovery-granularity emulations over the same underlying trajectory; we do not attribute their numbers to external artifacts. BranchFS, Waypoint, and other recent systems are discussed qualitatively because their dependencies are unavailable on this VM.

6.2 Runtime Overhead and Scaling

The current 64-call comparison is summarized in Table 2. Bare is not correctness-equivalent: it has a host pollution rate of one. Full AGENTTX is 43% faster than per-call isolation, but still $2.99\times$ the bare per-step latency. The gap between full and no-trace reflects both system-call tracing and the larger read-effect graph.

At lengths 54, 64, and 96, full AGENTTX costs 156.4, 139.5, and 107.0 ms per step; total wall time still grows from 8.45 to 10.27 s. The falling per-step number therefore indicates amortized fixed cost, not faster long workloads. Full-path step p50 is 19.6–24.4 ms, while p95 is 684.8–724.0 ms, showing that tests, traces, and state boundaries dominate the tail.

6.3 Causal Retention and Dependency Ablation

We execute real filesystem effect DAGs rather than simulate graph nodes. The sweep varies length (16/32/64), shape (chain/fan-out/layered), requested fault position (10/50/75%), and independent-work ratio (25/50/75%). Four modes receive the same declared reads: causal, temporal, whole-session, and causal without dependency capture. Three repeats yield 144 real-overlay runs over 48 aggregated configurations; all keep the host clean before commit.

Figure 3 shows that full causal rollback retains 100% of independent work and removes 100% of invalid descendants in every configuration. At 64 calls, temporal rollback removes invalid work but retains only 41% of independent work; whole-session discard retains 0%. The no-dependency abla-

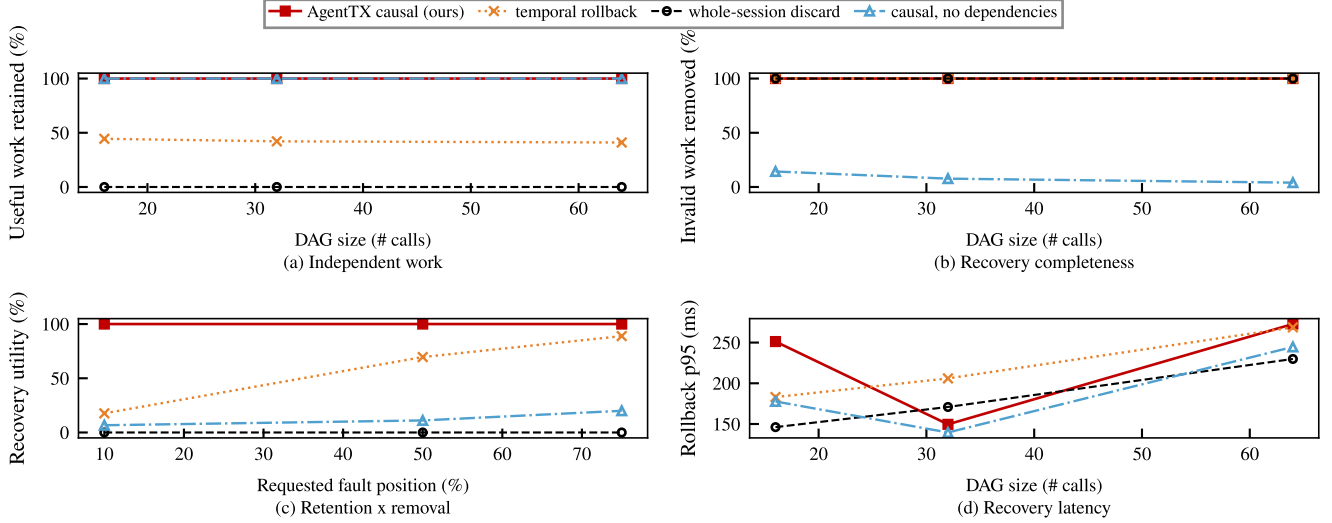


Figure 3: Causal recovery retains useful work and removes invalid descendants. At 64 calls, temporal recovery retains 41% of independent work and whole-session recovery retains none. Removing dependency capture preserves apparent work but removes only 4% of the invalid subgraph.

tion retains independent files but removes only 4% of invalid work. Thus selecting a small rollback set is not sufficient: dependency discovery and causal policy are jointly necessary. Causal rollback p95 at 64 calls is 272.7 ms.

6.4 Real-Agent Recovery

We seed a repository with a faulty pipeline, an independent release note, an artifact derived from the fault, and failing tests. A live LLM must inspect the ledger, select the earliest faulty producer, invoke causal rollback, retain the note, remove the artifact, repair the task, and pass tests. Across three fresh sessions, root selection, target selection, independent retention, invalid removal, commit, and tests succeed in all runs. The host leak rate is zero and wall p50/p95 is 29.0/30.8 s. This experiment validates that the control plane is usable by a real tool-selecting agent, but it is not yet a large-repository or multi-package benchmark.

6.5 Avoided Replay Tokens

The token study isolates model work caused by recovery granularity. A fixed five-step trajectory contains two valid documents separated by a faulty producer and its descendants. Causal rollback retains both documents; an optimistic temporal checkpoint loses one; whole-branch abort loses two. A real LLM regenerates only missing documents at 12, 24, and 48 entries, with three repeats per cell.

All 27 samples succeed with zero host leak. At 48 entries, temporal recovery replays 1,335.7 mean tokens and whole abort replays 2,891.0; AGENTTX avoids both costs because no valid document is discarded (Figure 4). These numbers are

avoided replay tokens, not total session-token savings, and the compared policies are granularity emulations rather than external systems.

6.6 Robustness and Resource Cost

An injected persistent-worker crash falls back to one-shot `try` for the current request and restarts the worker for the next. A 256-step session is closed at step 128, reloaded, completed, and commits all 256 files with step p50/p95 36.3/72.3 ms. Four concurrent agents, each writing 16 files in a separate workspace subtree, all commit without cross-contamination in 3.105 s. This test does not establish fencing for concurrent writes to the same path.

Content-addressed snapshots store 9.1 MB of unique blobs for 100.7 MB of logical snapshot payload (9.0%). This reduces repeated content but does not eliminate directory traversal, historical snapshot, or WAL-copy costs. Crash-injection tests also verify interrupted host materialization, persisted policies, native whiteouts, mode-000 trees, and metadata restoration.

7 Discussion and Limitations

Why not make causal rollback the default? The explicit API is correct for path hierarchy and symlink aliases, but lower hard links split during OverlayFS copy-up on our substrate [3]. Because a sibling name can observe stale content before ledger analysis, default-on causal recovery requires a different FUSE/kernel substrate or stronger object mediation. Bind-mount coverage is also incomplete.

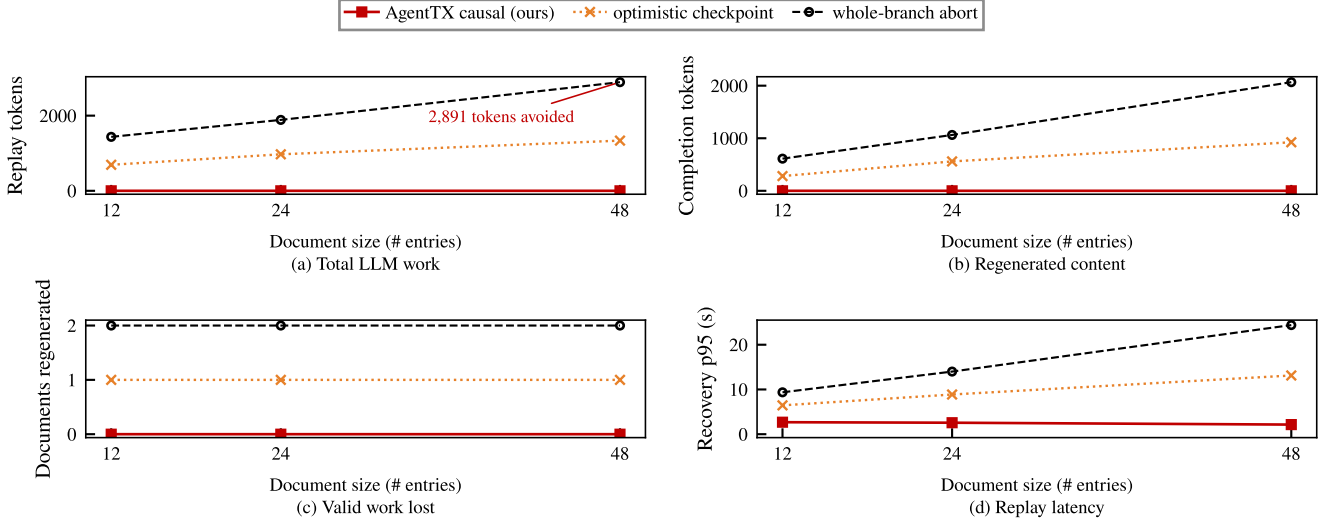


Figure 4: Real LLM replay caused by recovery granularity. Causal rollback retains both valid documents and requires no document-regeneration call. “Zero” excludes diagnosis, tests, rollback, and all tokens already consumed before failure.

Why trace instead of infer commands? Command strings are not a reliable specification of transitive compiler, test, or script reads. System-call tracing observes actual accesses but adds workload-dependent cost and remains Linux-specific. An eBPF implementation could reduce collection overhead and improve deployment, but would not remove the need to define alias and unsupported-syscall semantics.

What state is outside the transaction? The current AET covers workspace-local filesystem effects. Network requests, emails, package registries, cloud control planes, process memory, and terminal state may be irreversible or require subsystem-specific compensation. Systems that capture full process/session state are complementary, not dominated by AGENTTX.

Is commit atomic? The WAL guarantees deterministic recovery of the session and host after an interrupted commit. It does not provide kernel-level all-path atomic visibility to external observers during materialization. Workloads requiring that property need a rename-based publication protocol or a transactional substrate.

How strong is the current evidence? Deterministic workloads cover up to 256 steps and controlled DAGs up to 64 calls. Real-agent runs are seeded and use one model. Several recent external systems cannot run on the current VM because of FUSE, CRIU, kernel, compiler, or privilege requirements. A final submission needs broader repositories, interleaved statistical runs, and artifact-level comparisons where available.

8 Related Work

Effect capture and semisolates. `try` introduces unprivileged semisolates for opaque commands, including effect introspection, optional application, and stacking [2]. AGENTTX uses these mechanisms as its execution substrate. Its focus is the long-lived trajectory: automatic cross-step dependencies, non-contiguous causal recovery, frontier materialization, policy persistence, and commit WAL.

Agent-native filesystems and branches. YoloFS stages mutations, exposes snapshots, and uses progressive permission to improve agent/user control [8]. Its filesystem mediation and permission interface are broader than AGENTTX’s current substrate. BranchFS provides nested copy-on-write branch contexts with atomic leaf commit/abort and first-commit-wins coordination [5]. These systems select snapshots or branches; AGENTTX selects a causal subgraph inside one evolving trajectory.

Checkpoint and restore. Waypoint combines filesystem deltas with process, memory, shell, and terminal state for branchable sessions [7]. DeltaBox accelerates complete sandbox checkpoint/rollback using incremental filesystem and process state [1]. Crab observes turn-level effects to avoid checkpoints that do not contain recovery-relevant state [6]. Their state coverage or checkpoint efficiency is complementary to AGENTTX. AGENTTX instead answers which non-contiguous filesystem effects to remove after a failure has been identified.

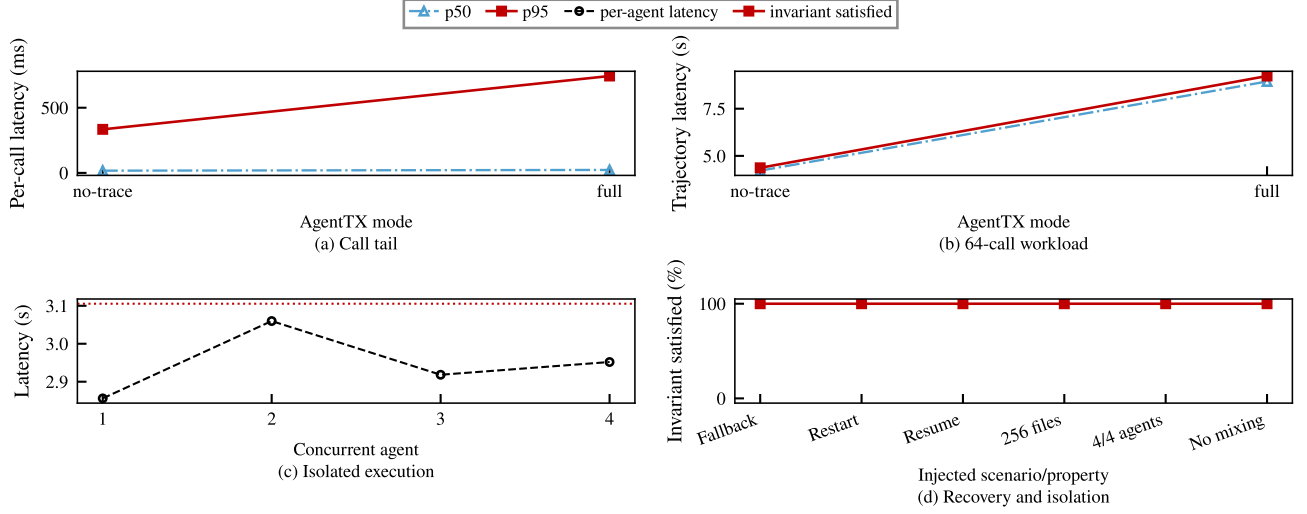


Figure 5: Tail, failure, long-session, and concurrency checks. Heterogeneous scenarios are kept in separate panels instead of collapsed into one score.

9 Conclusion

Long-running coding agents need a transaction boundary larger than one command and a recovery unit smaller than one session. AGENTTX represents a trajectory as a causal effect transaction: opaque tool effects form a dependency DAG, mutations remain speculative in a shared semisolate, causal rollback removes a failed producer and its descendants, and a durable commit frontier publishes approved effects. The prototype demonstrates perfect useful-work retention and invalid-descendant removal in the current controlled DAG suite, converts that retention into measurable replay-token savings, and survives worker crashes, session reload, and disjoint concurrency. The remaining agenda is equally clear: stronger object-identity semantics, lower tracing and snapshot overhead, non-filesystem compensation, and broader external baseline evaluation.

References

- [1] Yunpeng Dong, Jingkai He, Shiqi Liu, Yuze Hou, Dong Du, Zhonghu Xu, Si Yu, Baochuan Yang, Yubin Xia, and Haibo Chen. Deltabox: Scaling stateful ai agents with millisecond-level sandbox checkpoint/rollback, 2026.
- [2] Evangelos Lamprou, Tianyu (Ezri) Zhu, Di Jin, Grigoris Ntousakis, Georgios Liargkovas, Calvin Eng, Konstantinos Kallas, Michael Greenberg, and Nikos Vasilakis. Con-
- trolling Opaque-Component effects with semisolates and try. In *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26)*, pages 453–471. USENIX Association, 2026.
- [3] The Linux Kernel Developers. Overlay filesystem. Linux kernel documentation, 2026.
- [4] The strace developers. strace: Linux system call tracer, 2026.
- [5] Cong Wang and Yusheng Zheng. Fork, explore, commit: Os primitives for agentic exploration, 2026.
- [6] Tianyuan Wu, Chaokun Chang, Lunxi Cao, Wei Gao, and Wei Wang. Crab: A semantics-aware checkpoint/restore runtime for agent sandboxes, 2026.
- [7] Jiakai Xu, Tianle Zhou, Danielle Gillai, Ruizhe Fu, Georgios Liargkovas, Patrick Shen, Kostis Kaffes, and Eugene Wu. Toward systems foundations for agentic exploration. In *Systems for Agentic AI Workshop at SOSP*, 2025.
- [8] Shawn Wanxiang Zhong, Junxuan Liao, Jing Liu, Mai Zheng, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. Don’t let ai agents yolo your files: Shifting information and control to filesystems for agent safety and autonomy, 2026.